

Contents

The Aegis: A Post-Mortem	1
Abstract	1
1. The Problem	2
2. Design Choices	2
3. How It Works	3
4. Trial and Error	5
5. Numbers	8
6. Reflection	12

The Aegis: A Post-Mortem

Abdullah Hasan · Student ID 807271 Source code: github.com/Abdullah-373/The-Aegis

Abstract

The Aegis is a small web app for reading contracts. You drop a PDF, a small panel of AI agents argues about it, and you get a verdict — GO, NO-GO, or MAYBE — with a 0-to-100 risk score and a list of conditions to fix before signing.

The thing went through three versions. V1 was three prompt calls in a row: a Strategist talks the deal up, a Red Team tears it apart, a Judge writes the ruling. That worked but it was barely a “multi-agent system” — it was a chain. V2 rebuilt the middle of the pipeline on LangGraph and added a Planner that picks which specialists to run (Financial, Legal, Data, Compliance, Operations), gave the specialists a search_precedent tool to pull from a built-in knowledge base of 34 contract risk patterns, and added a critique step where the Strategist and Red Team get to push back on the Judge. V3 (the build this report is about) plugged OpenAI in next to Gemini so the same pipeline can be driven by gpt-5, gpt-5-mini, gpt-4o-mini, or any of the Gemini Flash/Pro tiers depending on which key you paste in.

Headline numbers from live runs:

- **GPT-5, balanced contract:** 617.03 s, 6,704 tokens, \$0.0941 list-price, CONDITIONAL-GO at risk 61.
- **GPT-4o-mini, same contract:** 100.54 s, 4,241 tokens, \$0.0014 list-price, CONDITIONAL-GO at risk 70. Same verdict tier as GPT-5, about 67× cheaper, 6× faster.
- **GPT-5-mini, mixed-risk contract:** 253.80 s, 6,072 tokens, \$0.0043, CONDITIONAL-GO at risk 78. 11 risks and 13 negotiated conditions in the output.
- **Cache replay on the GPT-5 row above:** 1.68 s, 0 tokens, \$0.00 — the SQLite cache saved the full 10-minute run on a re-upload of the same PDF.

Everything ran on free-tier or trial credit. The dollar numbers are list-price equivalents, not money out of pocket.

1. The Problem

Contracts are long. Nobody reads them. You either skim and miss the bad parts, or you hand the document to a lawyer for a few hundred dollars per page. Neither answer is good.

The obvious move is to throw an LLM at it and ask for a summary. That fails for two reasons. First, one model writing one summary always lands in the middle — it averages the upside and the downside into bland prose that does not actually help anyone decide whether to sign. The thing you want is the *spread*: the best possible reading of the deal *and* the worst possible reading, both at full strength, with a third opinion on top. Second, model output is free-form text. If you want a yes/no out of it, you end up parsing the prose with regular expressions, and that breaks the first time the model phrases its answer slightly differently.

The Aegis tries to fix both problems. It never gives you a neutral summary. It shows you the bullish case and the bearish case as two separate transcripts you can read in parallel, then a Judge agent reads both and emits a verdict as a JSON object with a fixed schema. Downstream code consumes the JSON, the human reads the transcripts.

There was a third problem that only showed up halfway through the build: a user who already pays for an OpenAI key should not need to also get a Gemini key just to use the app. V3 fixed that. Paste any supported key and the right provider is picked for you.

2. Design Choices

This section is the reasoning behind the building blocks, not a survey.

Three agents, not one. A single summary hides the gap between the optimistic and pessimistic reading. The whole point of a contract review is that gap. I split it into three agents with hard role boundaries: Alex (Strategist) only talks the deal up, Sam (Red Team) only attacks, Maya (Judge) reads both and rules. Fixed pipeline, not free-form chat. The transcripts stay auditable.

LangGraph for the V2 pipeline. Once V2 needed a Planner, five conditional specialists, two critique nodes, and an optional revision pass, hand-rolling the control flow in `main.py` got ugly fast. I moved it into a StateGraph with explicit nodes and conditional edges. Now the topology is one diagram instead of three pages of if statements.

One tool, not a tool-use framework. Specialists and the Red Team have access to exactly one tool: `search_precedent(query)`. It runs TF-IDF over 34 hand-written contract patterns in `knowledge_base.py` and returns the top 4. No vector store. No

embedding API call. About 60 lines of Python total. The corpus is small enough that this is faster than calling anything external.

Pydantic with Literal enums for the verdict. Maya is told to end her response with a fenced JSON block. The schema has `Literal["GO", "NO-GO", "CONDITIONAL-GO"]` for the verdict and `Literal["Low", "Medium", "High"]` for severity. If the JSON is wrong, validation fails loudly instead of silently accepting "high" as a valid severity. There are four fallback layers behind that for the times Maya gets it wrong anyway.

Provider routing by key prefix. V3 looks at the first few characters of the API key. AIza... is Gemini. sk-... is OpenAI. sk-ant-... is Anthropic, which I detect on purpose so I can reject it with a clear error instead of letting the user wait 10 seconds for an auth failure. The model picker in the UI groups models by provider and auto-switches when the detected provider changes. The user never picks "Provider" from a dropdown.

SQLite with a content-hash key. Cache key is `SHA-256(extracted_text + model_name)`. Three things follow. Renaming the PDF still hits the cache because the filename is not in the hash. Switching models forces a fresh run because the model name *is* in the hash. A one-character edit to the PDF text gives a completely different hash so the cache cannot get poisoned by an almost-identical file.

Lifespan, not on_event. Starlette has been warning about `@app.on_event("startup")` for a while. On Python 3.14 the `DeprecationWarning` was the first thing printed on boot, which made the app look unfinished even though nothing was broken. V3 moved the OCR-availability log line into a proper lifespan async context manager. Clean startup log.

Auto-open browser. The original quick-start was "run python main.py, then open `http://localhost:8000` in your browser." The first few times I ran V3 I forgot the second half, sat looking at the uvicorn log for a few seconds, and wondered why nothing was happening. So V3 schedules `webbrowser.open()` from the lifespan handler, one second after the server is ready. The env var `AEGIS_NO_BROWSER=1` skips it for Docker and SSH.

3. How It Works

The app is one FastAPI service. The layout is flat on purpose.

- `main.py` — FastAPI app, lifespan, WebSocket pipeline, the provider factory, the two mode branches (Fast / Full), retry and backoff, the structured-output recovery chain, and the cache write.
- `agents.py` — the LangGraph state machine, every node function, the tool-execution loop.
- `knowledge_base.py` — the 34 risk patterns and the TF-IDF retriever.
- `tools.py` — the LangChain @tool wrappers around the retriever.
- `database.py` — SQLAlchemy models with SQLite WAL mode.
- `templates/index.html` — one HTML file with three views (setup, live, verdict) and the past-reports drawer.

- tests/test_main.py — 33 unit tests.
- samples/ — three sample contracts (sample_contract.pdf, contract_balanced.pdf, contract_mixed.pdf).

3.1 The two modes

Setup has a toggle. **Fast** runs the original three calls — Alex, Sam, Maya — straight from the PDF. 3 API calls per analysis. Fits in the Gemini free tier with room to spare. **Full** runs the LangGraph pipeline below. 8 to 15 calls per analysis depending on what the Planner picks and how many times Sam reaches for the precedent tool. One or two of those a day on a free Gemini key. Effectively unlimited on a paid OpenAI key.

Both modes write to the same cache and emit the same verdict payload, so the verdict dashboard does not care which mode produced the answer.

[SCREENSHOT: Insert the **setup view** showing the model picker grouped by provider, the Fast/Full toggle, and the file drop zone. Caption: “Setup view — the model picker is grouped by provider; the picker auto-switches when the pasted key changes prefix.”]

3.2 A Full-mode run, end to end

Six phases.

1. **Planner.** Reads the document, returns a JSON list of two to five specialists from {financial, legal, data, compliance, operations}. One cheap call so we do not pay for specialists that have nothing to say.
2. **Specialists.** Each selected specialist runs in turn with access to search_precedent. Most of them call the tool one to three times. Each writes a short Markdown report with a severity rating.
3. **Alex.** Reads every specialist report and writes the strongest possible bullish case for the deal.
4. **Sam.** Reads the specialist reports *and* Alex’s case. Quotes Alex’s exact claims and attacks them. Calls search_precedent to ground attacks in documented patterns.
5. **Maya.** Reads everything. Writes a Markdown **## RATIONALE** section, then exactly one fenced JSON block at the end. The JSON has to validate against the Pydantic schema.
6. **Critique → optional revise.** Alex and Sam each respond **ACCEPT:** or **DISSENT:** to Maya’s ruling, in parallel via `asyncio.gather`. If at least one dissents, Maya runs once more with both critiques in context. If both accept, the original ruling stands.

[SCREENSHOT: Insert the **live tribunal view** with Alex, Sam, and Maya streaming side by side. Caption: “Live transcripts — Alex argues the bullish case (left), Sam quotes Alex and dismantles each claim (middle), Maya writes the rationale and the JSON ruling (right). Tokens arrive over a single WebSocket and animate into each card as they come in.”]

3.3 The structured-ruling recovery chain

Maya's prompt asks for a fenced JSON block at the end. Most of the time she complies. Sometimes she does not. The pipeline has four layers behind her.

1. A regex grabs the last fenced ````json` block in Maya's output. `json.loads` it. Validate against Pydantic with the `Literal` enums in place. If this works (and it almost always does) we are done.
2. If layer 1 fails, send Maya's full text *back* to the same model at `temperature=0` with the instruction "convert this narrative into clean JSON, output only the JSON." Parse and validate again.
3. If layer 2 fails, escalate to the larger model in the *same* provider — `gemini-2.5-pro` for Gemini runs, `gpt-4o` for OpenAI runs. The escalation never crosses providers, because the user only handed us one key.
4. If layer 3 still fails, a hand-written heuristic scans the text for the words `NO-GO`, `CONDITIONAL`, or `GO` and builds a default ruling with a synthetic risk score and four explanatory rows.

Layer 1 wins on every real run I logged. Layers 2 and 3 only fire when I deliberately feed broken JSON in. Layer 4 has never fired against the live model — it exists so the app cannot crash on the user.

3.4 The cache

One SQLite table. Key: `SHA-256(extracted_text + model_name)`. Stored on a cache hit: the three transcripts (Alex, Sam, Maya), the verdict, the risk score, the structured ruling, the input/output token counts, the list-price cost, plus the metadata flags (`truncated`, `chunked`, `critique_dissent`). Not stored: the API key.

A cache hit replays the run to the browser by drip-feeding the cached transcripts back through the WebSocket with a tiny 3-ms-per-64-chars delay, so the page animates in as if it were a fresh run. The actual SQLite lookup is sub-millisecond — the floor on a replay is the streaming delay, not the database.

[SCREENSHOT: Insert the **past-reports drawer** showing 3-4 cached rulings with model badges, risk scores, token counts, and timestamps. Caption: "Past reports drawer — click any card to re-open the full transcripts without re-running the pipeline. Delete icon clears the cache row."]

[SCREENSHOT: Insert the **verdict dashboard on a cache replay** — the one with `FROM CACHE · 617.03 s · SAVED 617.03 s`. Caption: "The headline measurement of the project: a 10-minute GPT-5 run replays from the SQLite cache in under two seconds. Cost on the replay is \$0.00."]

4. Trial and Error

The things that broke before they worked.

4.1 Maya would not produce clean JSON

The first version of Maya’s prompt asked nicely for JSON at the end and trusted her. That broke on the second real PDF I tested. The JSON had extra backticks wrapping it. Or a stray sentence after the closing fence. Or two JSON blocks, one as a worked example in the rationale and one as the real ruling. Or the word "high" where I needed "High". The first version just crashed on any of these and showed a 500 to the browser after waiting 30 seconds for the verdict. I noticed because I demoed it to myself, watched the timer tick, and got an HTTP error page.

The fix was the four-layer chain in §3.3. The dumb regex that grabs the *last* fenced block (not the first) was the single highest-value line of code in the whole project — it ate most of the “extra example block in the rationale” failures by itself.

4.2 The WebSocket kept burning my Gemini quota

WebSockets do not have a clean lifecycle like HTTP requests. I closed the browser tab during a Full run once, walked away, came back the next morning, and most of my daily Gemini free-tier quota was gone. The server had kept generating tokens into a socket nobody was reading.

The fix on the server side: every `_send(ws, ...)` call sits inside a natural error path now. When the socket drops, the next send raises `WebSocketDisconnect`, that exception bubbles up through the astream loop, and the in-flight Gemini call gets cancelled by the async runtime. The fix on the client side: the page has a close event listener that resets the UI back to setup if the disconnect happened mid-stream. There is also a Cancel button so users can explicitly stop a run instead of closing the tab.

4.3 LangGraph silently swallowed my coroutines

When I rebuilt the V2 pipeline on LangGraph, the first run came back with:

```
InvalidUpdateError: Expected dict, got <coroutine object _node_planner at 0x...>
```

The node was registered as a sync lambda returning a coroutine:

```
g.add_node("planner", lambda s: _node_planner(s, llm, emit))
```

LangGraph introspects each callable to decide whether to await its return value. A sync lambda that *returns* a coroutine does not register as an async function, so LangGraph passed the un-awaited coroutine straight to the state reducer, which complained it got a coroutine instead of a dict.

The fix:

```
def _bind(node_fn):
    async def _wrapped(state):
        return await node_fn(state, llm, emit)
    return _wrapped
```

```
g.add_node("planner", _bind(_node_planner))
```

A real `async def` registers as a coroutine function and `LangGraph` awaits it. I now know that “this lambda returns a coroutine” is not the same as “this lambda is `async`.”

4.4 Full mode burned through the daily cap in one run

Full mode does 8 to 15 calls per analysis. Gemini Flash free tier is 20 calls per day. Doing the math after the first time I tried Full on the free tier was depressing. One Full run, the rest of the day was dead. The next run died mid-stream with `RESOURCE_EXHAUSTED: 429`.

This is when I split the app into two modes. Fast is the default. Full is opt-in. I also rewrote the retry path: Google’s 429 errors include a `retryDelay` field, so the retry code now parses it and sleeps for the suggested delay instead of an arbitrary exponential backoff. If the error is the daily-cap variant (which retrying will never fix) I surface a clear message — “switch to a different model, switch to Fast, or wait for the daily reset” — instead of spinning through useless retries.

4.5 Tailwind silently dropped my hover colour

A small bug that took me embarrassingly long to find. Past-report cards have a hover border that matches the verdict colour. I wrote it as a template literal:

```
card.className = `card-soft p-4 hover:border-${accent}-200`;
```

where `accent` was one of “emerald”, “amber”, “rose”. Looked fine. The hover border never showed up on any card.

I spent an hour in the inspector before I figured it out. Tailwind’s JIT only emits a CSS rule for class names it can see as *literals* in the source. It never sees `hover:border-emerald-200` written out anywhere because I build the string at runtime. So the rule never gets emitted, the browser receives a class the stylesheet does not define, and the hover does nothing.

The fix is ugly but works:

```
const HOVER_BORDER = {
  'GO': 'hover:border-emerald-300',
  'NO-GO': 'hover:border-rose-300',
  'CONDITIONAL-GO': 'hover:border-amber-300',
};
card.className = `card-soft p-4 ${HOVER_BORDER[verdict]}`;
```

Lesson: any build step that depends on static analysis will not see strings I build at runtime.

4.6 The OpenAI switch quietly broke the JSON fallback

V3 introduced a regression I only noticed because of an auth error. Layer 3 of the recovery chain (§3.3) originally hard-coded `gemini-2.5-pro` as the escalation model. After I added OpenAI support, an OpenAI run that produced malformed JSON would silently try to recover against Gemini Pro — using a Gemini key the user had never

given us. The first time it happened I got an auth error against AIza... while running on sk-..., which was very confusing for about five minutes.

The fix is a per-provider map: {"google": "gemini-2.5-pro", "openai": "gpt-4o"}. The escalation reads the original key's provider and picks the larger model in the same family. A run never crosses providers now.

4.7 Python 3.14 refused to install pydantic

The most recent break. A reviewer tried to run the project on a fresh Python 3.14 install and `pip install -r requirements.txt` failed with a long Rust backtrace ending in `Failed building wheel for pydantic-core`. The cause: `pydantic==2.10.4` did not ship pre-built wheels for CPython 3.14 at that point, so `pip` tried to compile `pydantic-core` from source through `maturin` and `cargo`. Most Windows machines do not have a Rust toolchain.

Fix was a one-line bump: `pydantic>=2.11.0,<3.0`. From 2.11 there are pre-built wheels for 3.14. Install completes in seconds again without Rust in the picture.

4.8 @app.on_event("startup") made the boot log look broken

Same Python-3.14 install, second annoyance. Boot log printed:

```
DeprecationWarning: on_event is deprecated, use lifespan event handlers instead.
@app.on_event("startup")
```

The OCR-availability check used `@app.on_event("startup")`. Nothing was actually broken, but a `DeprecationWarning` on boot is a bad first impression. Migrated the handler into a real lifespan `async` context manager. Warning gone, log clean.

4.9 The app started but never opened in the browser

After the lifespan fix, the next run produced this:

```
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

The user sat staring at it, waiting for a tab to pop up. None came. The app had never been wired up to auto-open one; that line was always “open localhost:8000 yourself.” `0.0.0.0` is the *bind* address, not a URL you put in the browser. Two seconds of `webbrowser.open()` scheduled from the lifespan handler fixed it, with an `AEGIS_NO_BROWSER=1` env var to opt out for headless deployments. Small change, big quality-of-life win.

5. Numbers

What I measured against the live OpenAI and Gemini APIs on the V3 multi-agent pipeline.

5.1 The three test documents

Three PDFs in samples/, each picked to land somewhere different on the risk scale.

- `sample_contract.pdf` — two pages, deliberately hostile to the client. Five planted clauses: a 100% early-termination penalty, a 3-month liability cap, a perpetual royalty-free licence over anonymised client data, one-sided indemnification, and forced arbitration with a jury-trial waiver. Designed to land at NO-GO with a high score.
- `contract_balanced.pdf` — a well-drafted SaaS Master Licence. Mutual indemnification, a 2× liability cap with data and IP carve-outs, customer-owned data, a 99.9% SLA with automatic service credits, a 30-day cure period, a 90-day data-export window. Designed to land at GO or a soft CONDITIONAL-GO.
- `contract_mixed.pdf` — a marketing analytics agreement with a deliberate mix. A 6-month liability cap, a non-refundable annual prepayment, a 50% early-termination penalty, an anonymised-data licence that survives termination for two years, unspecified data residency. Designed to land at CONDITIONAL-GO with a moderate-to-high score.

5.2 Cross-model benchmark

Every row is a real Full-mode run on the live API. Cache disabled. Costs are list-price equivalents at the providers’ published per-million-token rates.

Document	Model	Wall-clock	Tokens	Cost	Verdict	Risk
<code>sample_contract.pdf</code>	gpt-4o-mini 2.5-flash	132.38 s	4,337	\$0.0057	NO-GO	95
<code>sample_contract.pdf</code> (run B)	gpt-4o-mini 2.5-flash	140.13 s	4,907	\$0.0071	CONDITIONAL-GO	88
<code>contract_balanced.pdf</code>	gpt-4o-mini	617.03 s	6,704	\$0.0941	CONDITIONAL-GO	61
<code>contract_balanced.pdf</code>	gpt-4o-mini	100.54 s	4,241	\$0.0014	CONDITIONAL-GO	70
<code>contract_mixed.pdf</code>	gpt-4o-mini	549.98 s	6,623	~\$0.0820 *	CONDITIONAL-GO	82
<code>contract_mixed.pdf</code>	gpt-4o-mini	253.80 s	6,072	\$0.0043	CONDITIONAL-GO	78

* GPT-5 on the mixed contract is the only row where I do not have the live dollar figure stored; that one is estimated from the per-token price. Every other row’s cost is the live figure from the cached verdict export. Two of those exports are committed under docs/sample_verdicts/.

[SCREENSHOT: Insert the **verdict card from the GPT-5 run on contract_balanced.pdf** — risk 61, 617.03 s, \$0.0941, model “gpt-5”. Caption: “GPT-5 on the well-drafted contract — CONDITIONAL-GO at risk 61. Nine pre-signature conditions including total financial commitment cap, strict data and AI usage limits, indemnity supercaps with insurance, and change-and-deprecation controls.”]

Three things stand out.

The verdict tier is robust across model families. Every OpenAI run on the balanced and mixed contracts came back CONDITIONAL-GO. The Gemini run on the deliberately hostile contract came back NO-GO. The agents disagreed on the *score* — 61, 70, 78, 82, 88, 95 across the rows — but they agreed on the *category*. The structured ruling is the only reason that variance is visible. A free-form summariser would have hidden it.

Mini-tier OpenAI models are stupidly cheap for the same verdict tier. GPT-5 on `contract_balanced.pdf` cost \$0.0941. GPT-4o-mini on the same document cost \$0.0014. That is about a 67× cost reduction for a ruling that landed in the same verdict band (CONDITIONAL-GO, scores 61 vs 70). For the “quick scan to decide if I need to read this myself” use case, GPT-4o-mini is the default-default.

Wall-clock is dominated by the strongest model. GPT-5 takes ~10 minutes per Full run. GPT-4o-mini does the same pipeline in 1:40. Same code, same number of LLM calls, same prompts. The bottleneck is per-call latency on the model, which compounds across Planner → 5 specialists → Alex → Sam → Maya → 2 critiques → optional Maya revision.

5.3 Cache replay

A second click on the same PDF and the same model returns from the SQLite cache.

Original run	Cache replay	Saved
GPT-5 on <code>contract_balanced.pdf</code> — 617.03 s, \$0.0941	1.68 s	full ten minutes, full \$0.0941
Gemini Flash on <code>sample_contract.pdf</code> — 30.65 s, \$0.0051	1.68 s	18× speedup, full \$0.0051

Zero API calls on a hit. The 1.68 s floor is the WebSocket round-trip plus the artificial token-streaming drip in `_replay_cached` (3 ms per 64 chars). The SQLite lookup itself is sub-millisecond.

[SCREENSHOT: Insert the **cache replay card** for the GPT-5 run — FROM CACHE · 617.03 S · SAVED 617.03 S pill, 6704 tokens, \$0.0000 next to a struck-through \$0.0941. Caption: “A 10-minute GPT-5 run replays in 1.68 seconds for \$0.00 of API spend. This is the single biggest measurable win in the project.”]

5.4 Non-determinism

I re-ran `sample_contract.pdf` twice in Full mode on Gemini Flash, with the cache disabled, to see how stable the verdicts are.

- Run A: NO-GO at risk 95, 132.38 s, \$0.0057.

- Run B: CONDITIONAL-GO at risk 88, 140.13 s, \$0.0071, with eight specific conditions.

Both runs found the same five core problems. They disagreed on the *tone* — walk away vs. fix and proceed. Three reasons:

1. Temperature is 0.2, not 0. I made that call early to keep the rulings from being completely robotic; the price is a small drift between adjacent verdict bands.
2. The Planner picks between two and five specialists. Different picks produce different syntheses.
3. Tool-call queries are model-chosen, so different KB entries come back on different runs.

Lowering the temperature would reduce the variance, not eliminate it. The cache eliminates it entirely for repeat queries on the same document, which is what the cache is for.

5.5 Verdict quality on the planted document

The five things I planted in `sample_contract.pdf` came out independently in the specialist reports:

- The Legal specialist flagged the 3-month liability cap and the one-sided indemnification.
- The Data specialist flagged the perpetual royalty-free licence as a privacy and competitive risk.
- The Operations specialist flagged the “commercially reasonable efforts” SLA as too vague to be enforceable.
- The Compliance specialist flagged the absence of a data-export mechanism on termination.

Maya’s mitigation language in the risk matrix maps almost verbatim onto specific KB entries. For the liability-cap row she wrote “*introduce carve-outs for data breaches, IP infringement, gross negligence, and willful misconduct*”. That is essentially the `liability_cap_zero_carveouts` entry in `knowledge_base.py`. The model is quoting documented patterns, not inventing them.

On `contract_balanced.pdf` the GPT-5 verdict softened to CONDITIONAL-GO at risk 61 — the agents did not invent risks that were not there, but they also did not give the document a clean GO because the auto-renewal clause and the data-licence carve-outs are *negotiable*, not *bad*. That is the behaviour the multi-document benchmark was designed to expose.

5.6 The test suite

33 unit tests pass. They cover content-hash determinism, JSON-block extraction with last-block precedence, Pydantic schema validation with invalid-severity rejection, heuristic-answer correctness across the three verdict bands, WebSocket setup rejection on missing key and bad model, the `/api/history` and `/api/verdict/{id}` endpoints, the chunker overlap arithmetic, transient-error classification, the cost calculator against the published per-million-token prices, the `/health` endpoint’s OCR-

availability flag, a seeded full-record fetch through `/api/verdict/{id}`, knowledge-base loading and retrieval relevance, tool registration and invocation, and the Lang-Graph topology.

6. Reflection

A small project that grew teeth as I figured out what was actually missing.

What worked

The four-stage JSON recovery chain. The single biggest engineering win. It turned the system from “crashes when the model misbehaves” into “always returns something.” Most LLM tutorials I have read pretend this problem does not exist. It does. Handling it was harder than building the rest of the pipeline.

The content-hash cache. About fifteen lines of SQLAlchemy and one SHA-256 call. Returns a 10-minute GPT-5 run in 1.68 seconds for \$0.00. The same idea would slot into almost any LLM app whose inputs are deterministic — I now think a content-hash cache should be the first thing you write, not the last.

The two-mode design. Fast and Full feel like a real product decision rather than a hack. The same codebase works on a free Gemini key and on a paid OpenAI key without compromising either path. The mode toggle is one line in the UI and a couple of branches in the pipeline.

The provider abstraction. Smallest cleanly isolated change in the whole project. Two functions (`detect_provider`, `model_provider`), one factory (`_make_llm`), one pricing table, one fallback ladder. That is all the diff between “Gemini demo” and “supports two of the three major model families.” Adding Claude would be one more prefix and one more factory branch.

What I would do differently

1. **Temperature 0 from day one.** The non-determinism in §5.4 was avoidable. I traded variance for a slightly more interesting voice, and the variance hurt more than the voice helped.
2. **Provider abstraction from day one, not in V3.** Hard-coding ChatGoogleGenerativeAI early made the OpenAI introduction in V3 more invasive than it should have been. `_make_llm` exists now and works, but the right time to write it was V1.
3. **Session cookie for the API key.** Users get annoyed pasting the key on every page load. The “we never store it” copy on the page does not actually make that less annoying.
4. **PostgreSQL from the start.** SQLite is great for solo use, but the single-writer constraint is going to bite the moment more than one person uses the app at once. The SQLAlchemy abstraction makes this a one-line change. I should have just done it.

5. **pydantic $\geq$ 2.11 and a lifespan handler from the start.** Both of the V3-era fixes (§4.7 and §4.8) were one-line changes I should have written correctly the first time. The cost of writing them right was nothing. The cost of fixing them after a reviewer hit them was a small but real chunk of my last weekend.

What is genuinely missing

Honest list of things that are not done.

- **No authentication.** `/api/history` and `/api/verdict/{id}` are open. On a shared deployment anybody with the URL reads every cached verdict in the database. This blocks any public deployment without an auth layer in front.
- **OCR is opt-in and the dependency story is bad.** Pure-text PDFs work out of the box. Scanned PDFs need `pytesseract`, `pdf2image`, and the `tesseract` and `poppler` *binaries* installed on the host. On Windows that is a manual download from a SourceForge mirror, which is exactly the kind of step nobody actually does. I documented it. I did not bundle it.
- **Agents pass state through the reducer, not to each other.** A Specialist cannot ask another Specialist a follow-up question. Sam cannot interrupt Alex mid-sentence. The “tribunal” framing in the marketing copy is louder than the actual inter-agent communication, which is a one-way state pass. A real multi-agent debate is a bigger project than what I shipped.
- **Cost numbers drift.** The per-token prices are hard-coded in `MODEL_PRICES`. If Google or OpenAI change their published rates, the dollar figures the dashboard shows the user become wrong silently. There is no automated price refresh.
- **Anthropic keys are detected but not supported.** `sk-ant-...` returns a clear error instead of working. Wiring Claude into the provider abstraction is one prefix and one factory branch — I did not have a Claude key during development. This is the obvious next addition.
- **The cache key includes the model name, which is exactly correct, except** when you only want to know whether the same document has been seen before regardless of model. There is no “have I ever seen this PDF?” lookup. Adding one is small but I have not done it.
- **The risk score is a model-emitted integer between 0 and 100, with no rubric.** The model picks a number it likes. The variance between runs ($95 \rightarrow 88 \rightarrow 70 \rightarrow 61$ across models on contracts that should mostly land in the high-medium band) shows the score is roughly calibrated but not really *calibrated*. A scoring rubric prompt would help. I did not write one.
- **The Past Reports drawer is per-instance, not per-user.** There is no concept of “your” reports vs “everyone’s” because there is no user system. Fine for solo use, completely wrong for sharing the URL with anyone.

Closing

Contract review is a domain where the *gap* between the optimistic and pessimistic readings matters more than the average of the two. The Aegis tries to make that gap visible instead of hiding it inside a single summary. The route from V1 (three sequential calls with role personas) to V2 (a LangGraph state machine with a Planner,

specialists, tool use, RAG, and a critique loop) to V3 (the same machinery driven by either Gemini or OpenAI, with cache-replay measurements that close the empirical gap from V2) is the real story of the project. All three versions ship in the same app behind a mode toggle because all three have a place. The cache makes the question of which combination you used irrelevant on a repeat query — the answer is in the database, and SQLite returns it in under two seconds.